

Tyler Vander Mooren

CYB 499: LLM and Cyber Security Applications

Professor Hayrettdin Bahsi

May 1, 2026

Final Project

Evaluating Large Language Models for Malicious Activity Detection in Windows Event Logs

Introduction and Motivation

Modern enterprise systems generate large volumes of endpoint telemetry through Windows Event Logs, Security logs, PowerShell logs and Sysmon logs. These logs are important for detecting suspicious behavior such as process execution, credential access, persistence, command-and-control activity and lateral movement. Sysmon is especially valuable because it records detailed system activity such as process creation, network connections and file-related activity in the Windows event log. However, the same visibility that makes Windows logs useful also creates a major analysis problem. Security analysts must interpret event identifiers, command-line arguments, parent-child process relationships, user context, registry changes, network destinations and script content. A single suspicious action may appear across multiple events, while individual events can look normal when viewed in isolation. This makes manual log analysis time consuming and difficult, especially when analysts must quickly decide whether a log entry represents benign system activity or malicious behavior. Large language models are useful candidates for this type of task because they can process structured and semi-structured text, interpret command-line syntax and produce natural-language explanations. At the same time, their reliability in cybersecurity detection workflows cannot be assumed. A model that

over-classifies benign logs as malicious may increase alert fatigue, while a model that under-classifies suspicious logs may miss real threats. For that reason, the analysis treats LLMs as possible classification aids, not as replacements for SIEM rules, detection engineering or analyst judgment.

Objective and Scope

The main objective is to evaluate whether LLMs can classify structured Windows Event Log entries as either Malicious or Benign. The evaluation also examines where the models fail and whether structured prompting improves performance compared with direct classification.

Four experimental conditions were tested:

- OpenAI GPT-5.4 API model with direct classification
- OpenAI GPT-5.4 API model with structured reasoning
- Qwen3-4B-Instruct-2507 with direct classification
- Qwen3-4B-Instruct-2507 with structured reasoning

The original proposal planned to compare an OpenAI model with a Llama model. During implementation, access to the selected Llama repository required gated Hugging Face approval and access was only granted for a different model. To preserve the original benchmarking design, the open weight comparison model was changed to Qwen3-4B-Instruct-2507. Qwen3-4B is a public instruction tuned language model that was practical for local testing and strict JSON classification tasks. The scope excludes fine tuning, multiclass attack type classification, full SIEM integration and production deployment. Each model receives one structured event at a time rather than a full timeline of related events. This keeps the evaluation measurable, but it also limits the amount of attack context available to the models.

Dataset

The dataset came from atomic-evtx, which provides Windows Event Logs generated from simulated attack scenarios on a Windows 10 system. It includes logs containing Windows sources such as Sysmon, Security, System, PowerShell and Application logs. Dataset validation showed that the original log collection was too large for direct manual review or full log LLM input. The benign activity logs alone contained tens of thousands of events, including 48,125 Sysmon events, 32,761 Security events and 820 PowerShell events. The malicious side also contained many scenario files, but not every event inside an attack scenario was relevant. Some attack folders included normal background system activity, including Microsoft Edge update behavior, SecurityHealthService activity and generic helper process activity. Because of that, the raw logs were converted into a controlled event level benchmark. The final evaluation dataset contains 100 total rows, split evenly between 50 benign events and 50 malicious events. Benign samples were selected from benign Sysmon, PowerShell and Security logs. Malicious samples were selected from attack scenario logs only when the individual event contained direct attack evidence or preserved parent command line context. This avoided the weak assumption that every event inside an attack folder should automatically be labeled malicious.

Table 1

Source distribution across final dataset

Label / Source	Count
Malicious / Sysmon	26
Malicious / PowerShell	24
Benign / Sysmon	20
Benign / PowerShell	15
Benign / Security	15

The malicious samples came from multiple attack behavior categories, including persistence, lateral movement, command and control, credential access, discovery, execution, collection and defense evasion. The most represented malicious scenarios were persistence using BITSAdmin, lateral movement using PowerShell and Excel COM behavior, command-and-control-style DNS/HTTPS query volume and LDAP credential brute force activity.

Benchmarking Design

The evaluation tested two models against the same 100 event records. The GPT-5.4 was accessed through the OpenAI API, while Qwen3-4B was run locally through a Python evaluation script. Both models received the same event summaries, the same ground truth labels and the same two prompt styles.

The benchmark used two prompt styles. The direct classification prompt gave the model the event record and asked it to classify the event as either Malicious or Benign with a short evidence statement. This prompt intentionally provided minimal guidance so the model had to decide which fields mattered.

```
{  
  "classification": "Malicious or Benign",  
  "confidence": "High, Medium or Low",  
  "evidence": "Brief explanation"  
}
```

The structured prompt used the same event record but gave the model a more explicit review process. It instructed the model to consider security-relevant fields such as event ID, provider, image, command line, parent image, user, registry fields, network fields and script content when available. It also required the model to identify the key fields that influenced the decision. The goal was not to generate long chain-of-thought reasoning, but to test whether directing the model's attention toward important log fields improved classification performance.

```
{
```

```
"classification": "Malicious or Benign",
"confidence": "High, Medium or Low",
"key_fields": ["field names used"],
"evidence": "Brief explanation"
}
```

All model outputs were saved to CSV files and evaluated with a Python metrics script.

The evaluation measured accuracy, precision, recall, F1-score, false positives, false negatives, true positives, true negatives and confusion matrices.

For the evaluation:

- **Malicious** was treated as the positive class.
- **Precision** measured how often malicious predictions were malicious.
- **Recall** measured how many true malicious events were detected.
- **False positives** represented benign events incorrectly flagged as malicious.
- **False negatives** represented malicious events incorrectly classified as benign.

Results

Table 2

Final benchmark results

Model / Prompt	Accuracy	Precision	Recall	F1-score	False	False
					Positives	Negatives
OpenAI-Direct	0.83	1.000	0.660	0.795	0	17
OpenAI-Structured	0.86	1.000	0.720	0.837	0	14
Qwen-Direct	0.82	0.944	0.680	0.791	2	16
Qwen-Structured	0.79	0.854	0.700	0.769	6	15

The best overall condition was the OpenAI structured prompt. It achieved 86% accuracy, 1.00 precision, 0.72 recall and a 0.837 F1-score. It also produced no false positives, meaning every benign event was correctly classified as benign. The OpenAI direct prompt also achieved perfect precision, but it missed more malicious events. Its recall was 0.66 compared with 0.72 for

the structured prompt. This means the structured prompt helped GPT-5.4 detect more malicious events without increasing false positives. Qwen direct performed competitively with 82% accuracy and a 0.791 F1-score, which was close to OpenAI direct. However, unlike OpenAI, it produced two false positives. Qwen structured slightly improved recall from 0.68 to 0.70, but this improvement came at the cost of additional false positives. Its false positives increased from two to six, precision dropped from 0.944 to 0.854 and overall F1-score dropped from 0.791 to 0.769. The results show that structured prompting was helpful for GPT-5.4 but not universally beneficial. For Qwen, the structured prompt made the model more aggressive in identifying malicious behavior, but that also caused it to incorrectly flag more benign activity.

Analysis of Findings

The strongest finding is that the OpenAI structured prompt produced the best balance between malicious detection and false-positive control. This is important because a security tool that produces too many false positives can increase analyst workload, while a tool that misses malicious events can create detection gaps. In this benchmark, the OpenAI structured condition avoided false positives while improving recall compared with the direct prompt. The second major finding is that both models were conservative in different ways. OpenAI was especially conservative because it produced zero false positives in both prompt conditions. This is useful from an alert-fatigue perspective, but it also meant that malicious events were sometimes classified as benign. The OpenAI direct prompt missed 17 malicious events, while the OpenAI structured prompt missed 14.

Qwen showed a different pattern than OpenAI. It detected a similar number of malicious events compared with OpenAI direct, but it was more willing to classify benign events as malicious. This created false positives, especially in the structured prompt condition. That

behavior suggests that smaller open-weight models may be more sensitive to security-related terms or administrative activity that appears suspicious but is not actually malicious in context. The third major finding is that malicious event detection was easiest when the event contained a clearly suspicious command line, script block or process relationship. Events involving LDAP credential access logic, suspicious PowerShell behavior, WMIC uninstall commands and password-protected archive creation were more likely to be identified correctly. The models struggled more when the malicious meaning depended on a sequence of related events. For example, BITSAdmin persistence behavior may appear across multiple commands such as job creation, resume, completion and notification execution. Individually, some of these events can look like normal administrative activity. The model often needed the broader sequence to understand why the event was malicious.

Error Analysis

The most important false negative pattern involved dual use Windows utilities. These are legitimate tools that can be used by administrators or abused by attackers. Examples included:

- bitsadmin.exe
- chcp
- wmic
- conhost.exe
- WinRAR command-line usage
- PowerShell commands with limited visible context

Persistence events involving BITSAdmin were the most consistent source of missed malicious activity. Commands such as job creation, resume and completion were sometimes treated as benign because the model saw them as normal Windows administrative behavior. However, in the dataset, these events belonged to a persistence and download-execute chain. Discovery events involving chcp were missed too. A code page query can be a normal

administrative command, but it can also support system language discovery. When viewed as an isolated event, the command did not always contain enough context for the model to confidently classify it as malicious. Some command-and-control-adjacent events involving conhost.exe were also missed. This is understandable because conhost.exe is a normal Windows process. Without enough surrounding process-tree or timeline context, the model tended to classify it as benign. Qwen's false positives showed the opposite problem. It sometimes classified benign activity as malicious when the event contained security-relevant or administrative-looking behavior. This included benign update activity, PowerShell commands or normal background processes that appeared suspicious when viewed without broader context. Overall, the error analysis suggests that event-level classification is useful but incomplete. LLMs perform better when a single event contains enough evidence. They are weaker when the correct classification depends on reconstructing a multi-event story.

Limitations

Several limitations affect how the results should be interpreted. First, the dataset was intentionally small and controlled. The benchmark used 100 total events, which made manual validation and error analysis practical, but it is not large enough to represent all Windows endpoint activity. Second, the dataset was balanced with 50 benign and 50 malicious events. Real security environments usually contain far more benign activity than malicious activity, so the false positive impact in production could differ from this controlled result. Third, the project used event-level classification. Each model classified one event at a time. This design made the experiment clean and measurable, but it limited the model's ability to understand multi-event attack sequences. Many real attacks are easier to identify by looking at process chains, timelines, repeated commands or related events across multiple logs. Fourth, the malicious dataset was not

evenly distributed across all attack scenarios. Some scenarios contributed to more usable attack-relevant events than others. For example, persistence, lateral movement, command and control and credential access contributed more rows than Word DDE or defense evasion. This means the benchmark should be interpreted as a controlled sample of multiple malicious behaviors, not as a complete MITRE wide evaluation. Finally, the results only show how the models performed on a controlled static dataset.

Overall, the evaluation shows that LLMs can classify structured Windows Event Log entries with useful accuracy, but their performance depends heavily on model choice, prompt design and available context. The OpenAI structured prompt produced the strongest result, achieving 86% accuracy, perfect precision, 0.72 recall and a 0.837 F1-score. Structured prompting improved OpenAI performance by increasing malicious detection without introducing false positives. However, the same prompt design did not improve Qwen in the same way. Qwen direct performed competitively, but Qwen structured produced more false positives and a lower overall F1-score. This shows that structured prompting is useful, but not automatically better across every model. The most important limitation was context. Both models struggled when malicious behavior depended on a sequence of related events rather than a single obvious command line or process relationship. BITSAdmin persistence, CHCP discovery and other dual use administrative activity were harder to classify because the individual events could appear normal when separated from the larger attack chain.

This matters for cybersecurity because real intrusion detection rarely depends on one perfect log entry. Analysts usually build confidence by connecting process activity, user context, command history, network behavior and timing. These findings suggest that LLMs may be valuable as analyst support tools for log triage, explanation and prioritization, but they should not

replace detection rules, sequence-based analytics or human review. The strongest use case is not fully automated malicious activity detection. A more realistic role is helping analysts interpret suspicious logs faster, explain why certain fields matter and surface events that deserve closer review. Future work should evaluate larger datasets, sequence level event windows and hybrid workflows that combine LLM interpretation with traditional detection engineering. In their current state, these models should not be placed in full control of malicious activity detection. Both models scored below 87% accuracy, and even a small false negative rate can be unacceptable when missed malicious activity may allow an intrusion to continue. Their safer role is analyst support, not autonomous decision-making.

References

arniki. *atomic-evtx dataset*. GitHub. <https://github.com/arniki/atomic-evtx>